

CreditRisk AI: Project Documentation & Presentation Guide

Explainable Machine Learning Engine for Loan Default Risk Assessment
German Credit Dataset (UCI ML Repository) | Repository: NISHAL2007/S5_loan_default_prediction_

1. Executive Summary & Project Objectives

Financial institutions face significant capital risk when issuing credit to borrowers. Traditional credit scoring relies on rigid rule-based models or black-box machine learning algorithms that lack explainability. CreditRisk AI is an end-to-end, production-grade credit default risk prediction application built on historical credit data to address three core objectives:

- **Rigorous Predictive Modeling:** Evaluate classification algorithms (Logistic Regression, Random Forest, XGBoost) using cost-sensitive credit risk evaluation metrics (prioritizing Recall and ROC-AUC over misleading raw Accuracy).
- **Explainable AI (XAI):** Provide granular, applicant-specific feature attribution using SHAP (SHapley Additive exPlanations) log-odds contribution weights mapped accurately to raw applicant attributes without claiming non-existent physical causality.
- **Transparent Decision Governance:** Allow financial underwriters to dynamically adjust credit policy decision thresholds ($0.30 \leq \text{threshold} \leq 0.70$) and stress-test predictions under numerical perturbations using a Stability Analysis Lab and Counterfactual Engine.

2. Dataset Architecture & Integrity Specifications

The system is built on the historical German Credit Dataset (UCI Machine Learning Repository), adhering strictly to dataset integrity standards:

- **Total Sample Size (N):** 1,000 credit applicant records.
- **Predictor Variables:** 20 feature columns (7 numerical continuous/discrete features, 13 categorical features).
- **Target Variable (target):** Binary indicator where 0 = Good Credit (700 applicants, 70%) and 1 = Bad Credit / Default (300 applicants, 30%).
- **Data Splitting Strategy:** Stratified 80/20 train/test split (800 train records / 200 test records) using `random_state=42` and `stratify=y`.
- **Zero Data Leakage:** Preprocessing transformations (imputation, scaling, one-hot encoding) are fitted strictly on the 800 training samples.

Predictive Features Inventory (20 Attributes)

Feature Name	Type	Description / Value Categories
checking_status	Categorical	<0 DM, 0<=X<200 DM, >=200 DM, no checking
duration	Numerical	Credit duration in months (6 to 72 months)
credit_history	Categorical	critical/other existing credit, existing paid, delayed previously, no credits/all paid
purpose	Categorical	car (new/used), furniture/equipment, radio/tv, appliances, repairs, education, business, etc.
credit_amount	Numerical	Credit amount in historical currency (DM)
savings_status	Categorical	<100 DM, 100<=X<500 DM, 500<=X<1000 DM, >=1000 DM, no known savings
employment	Categorical	unemployed, <1 year, 1<=X<4 years, 4<=X<7 years, >=7 years
installment_commitment	Numerical	Installment rate in percentage of disposable income (1% to 4%)
personal_status	Categorical	Sex & marital status combinations
other_parties	Categorical	none, co-applicant, guarantor
residence_since	Numerical	Present residence duration in years (1 to 4 years)
property_magnitude	Categorical	real estate, building society savings/life insurance, car/other, no property
age	Numerical	Borrower age in years (19 to 75 years)
other_payment_plans	Categorical	bank, stores, none
housing	Categorical	rent, own, for free
existing_credits	Numerical	Number of existing credits at this bank (1 to 4)
job	Categorical	unemployed/unskilled non-resident, unskilled resident, skilled employee, management
num_dependents	Numerical	Number of maintenance dependents (1 or 2)
own_telephone	Categorical	none, yes
foreign_worker	Categorical	yes, no

3. Machine Learning Evaluation & Benchmark Matrix

In loan underwriting, a False Negative (approving a borrower who defaults) results in severe direct capital loss (loss of principal). Conversely, a False Positive (rejecting a good borrower) costs only lost interest income. Therefore, raw accuracy is misleading.

Model Name	Accuracy	Precision	Recall	F1-Score	ROC-AUC	PR-AUC	Composite Score
Logistic Regression (SELECTED)	0.7500	0.5581	0.8000	0.6575	0.8058	0.6215	0.8029
Random Forest Classifier	0.7700	0.6167	0.6167	0.6167	0.8039	0.6212	0.7103
XGBoost Classifier	0.7400	0.5690	0.5500	0.5593	0.7569	0.5926	0.6535

4. Mathematical Framework & Equations

1. Model Predicted Default Probability Equation:

$$P(\text{Default} = 1 \mid X) = 1 / (1 + \exp(-(W^T * X_{\text{transformed}} + b)))$$

2. Decision Classification Rule:

Decision = REJECTED if $P(\text{Default} = 1 \mid X) \geq \text{Threshold } (\theta)$

Decision = APPROVED if $P(\text{Default} = 1 \mid X) < \text{Threshold } (\theta)$

3. Composite Selection Score Formula:

$$\text{Model Selection Score} = (0.50 * \text{Recall}) + (0.50 * \text{ROC-AUC})$$

4. Probability Calibration (Brier Score):

$$\text{Brier Score} = (1 / N) * \sum (p_{\text{predicted}} - y_{\text{actual}})^2$$

Uncalibrated Brier Score: 0.1824 → Platt Calibrated Brier Score: 0.1562 (+14.3% boost)

5. System Audit & Bug Resolution Summary (Priorities 1 to 9)

Priority	Audit Description	Root Cause Analysis	Resolution & Implementation
Priority 1	Decision Correctness	Stale state or missing bounds	Enforced strict probability $\geq$ threshold -> REJECTED with NaN validation fallback.
Priority 2	SHAP Feature Mapping	Bug: <code>fname.split('_')[0]</code> mapped <code>credit_history</code> -> <code>credit</code>	Implemented <code>get_parent_feature_key()</code> in <code>src/explainability.py</code> matching exact <code>num_cols</code> & <code>cat_cols</code> .
Priority 3	Why This Decision Card	Missing factor breakdown	Created <code>ExplanationCard.tsx</code> rendering margin, top 5 risk factors, and top 3 protective factors.
Priority 4	Currency Label Cleanup	USD (\$) label on DM currency	Replaced 'Credit Amount (\$)' with 'Credit Amount' across all backend and frontend views.
Priority 5	Selection Transparency	UI text misstated recall alone	Updated text to match exact code: 'Model selection criterion: 50% Recall + 50% ROC-AUC'.
Priority 6	Stability Lab	Sensitivity lab showed NaN%	Validated predictions with <code>np.isfinite()</code> & renamed score to 'Project Stability Score'.
Priority 7	Probability Calibration	Raw probability lacked calibration	Added Brier score benchmarking (0.1824 -> 0.1562) & distinguished discrimination from calibration.
Priority 8	Full Applicant Test	Need automated test set check	Created <code>scratch/run_full_validation.py</code> auditing 30 test applicants (30/30 passed with 0 mismatches).
Priority 9	Dataset Integrity	Verify 1,000 records & 20 features	Validated 1,000 original German credit records without synthetic alteration.

6. Step-by-Step Viva Presentation Script for Project Guide

Step 1: Problem Statement & Objectives (1 Minute)

"Good morning/afternoon, Respected Guide. Today I am presenting CreditRisk AI, an explainable machine learning credit underwriting system built on the German Credit dataset (1,000 applicant records, 20 predictor attributes). In credit risk assessment, machine learning models often suffer from two major flaws: first, optimizing for raw accuracy while ignoring asymmetric default costs, and second, operating as 'black boxes' without trustworthy explanations. Our project solves both problems."

Step 2: Preprocessing & Model Selection Rationale (1 Minute)

"We split the dataset into an 80% training set and a 20% test set using stratified sampling to maintain the 70/30 good-to-bad credit distribution. To prevent data leakage, our imputation, scaling, and one-hot encoding pipelines were fitted strictly on training data."

We evaluated Logistic Regression, Random Forest, and XGBoost. Instead of picking models based on accuracy, we used a composite selection metric: 50% Recall + 50% ROC-AUC. Missing a defaulter costs principal capital, whereas rejecting a good borrower costs only lost interest. Logistic Regression achieved an 80.00% Recall and 0.8058 ROC-AUC, making it our selected model."

Step 3: Explainable AI & SHAP Bug Resolutions (1.5 Minutes)

"For model explainability, we integrated SHAP (SHapley Additive exPlanations). A critical bug in standard explainability scripts is using naive string splitting like `split('_')[0]`, which incorrectly maps `credit_history` to `credit` and collides with `credit_amount`. We fixed this by building a safe parent feature mapping function that aggregates one-hot encoded SHAP values back to original features."

Furthermore, our explanations strictly avoid claiming causality—instead of saying 'this caused default', our engine states 'this feature contributed toward higher model-estimated default risk'."

Step 4: Web Application & Underwriting Features (1 Minute)

"Our full-stack web application features:

- 1. Risk Assessment Engine: Real-time evaluation with dynamic credit decision threshold sliders (0.30 - 0.70) and a visual Decision Logic Flow (Probability $\geq$ Threshold $\rightarrow$ Decision $\rightarrow$ Risk Tier).*
- 2. Stability Analysis Lab: Stress-tests model sensitivity under +/- 5% numeric perturbations.*
- 3. Counterfactual Analysis Engine: Generates minimal attribute modifications needed to turn a rejection into an approval."*

Step 5: Verification Audit & Conclusion (30 Seconds)

"We conducted a full 30-applicant automated correctness audit (`scratch/run_full_validation.py`). All 30 test applicants demonstrated 100% agreement between backend probability, threshold, frontend decision, and SHAP feature mapping, with zero NaN values. The application is deployed live on Vercel."